Object Oriented Software Engineering

Chapter Five

System Design

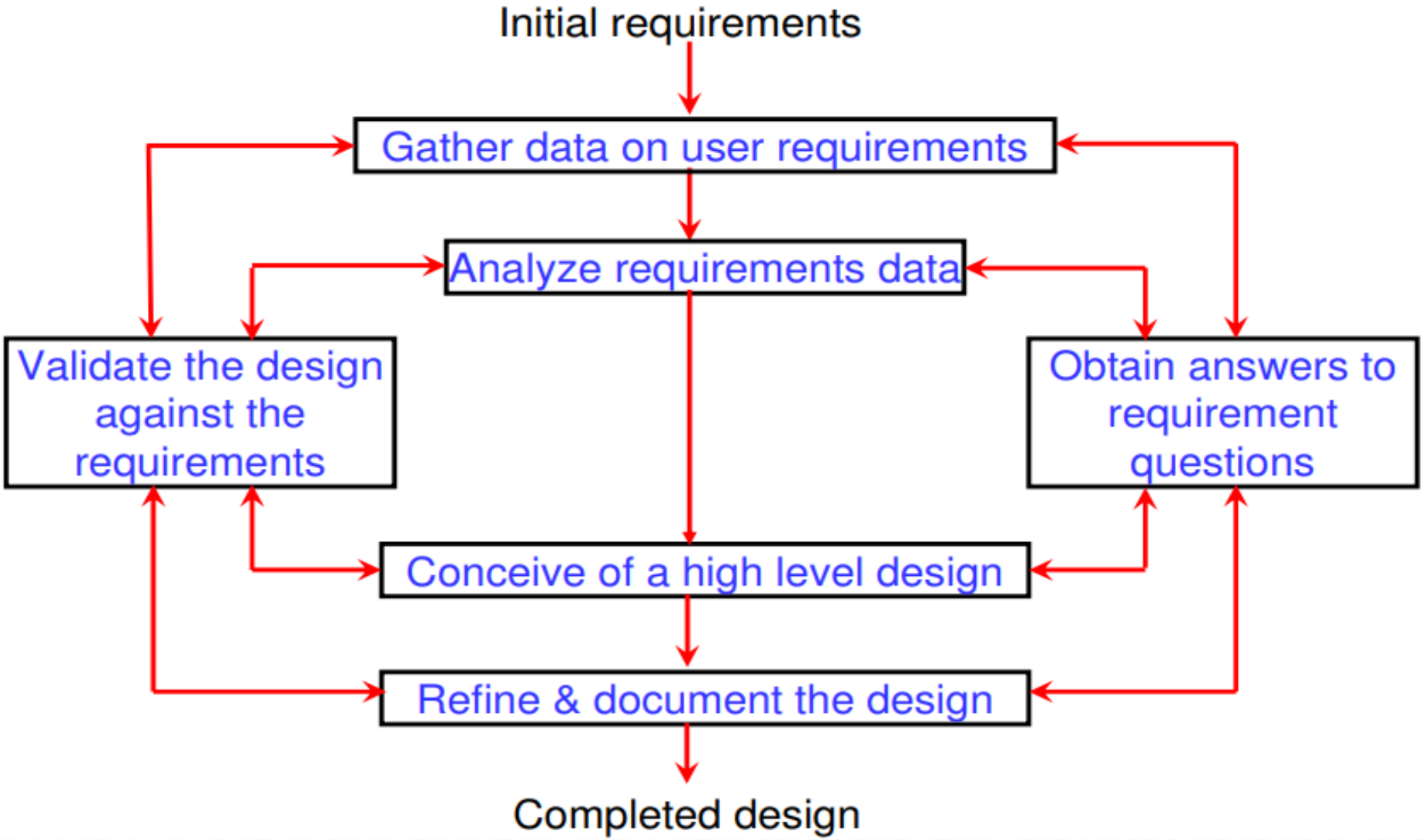
Outlines

- Overview of system design
- System Design Strategies
- System Design Approach
- Object Oriented Design
  - Identify design goals
  - Design the initial subsystem decomposition
  - Cohesion and Coupling
- *Reviewing System Design*
- *Managing System Design*

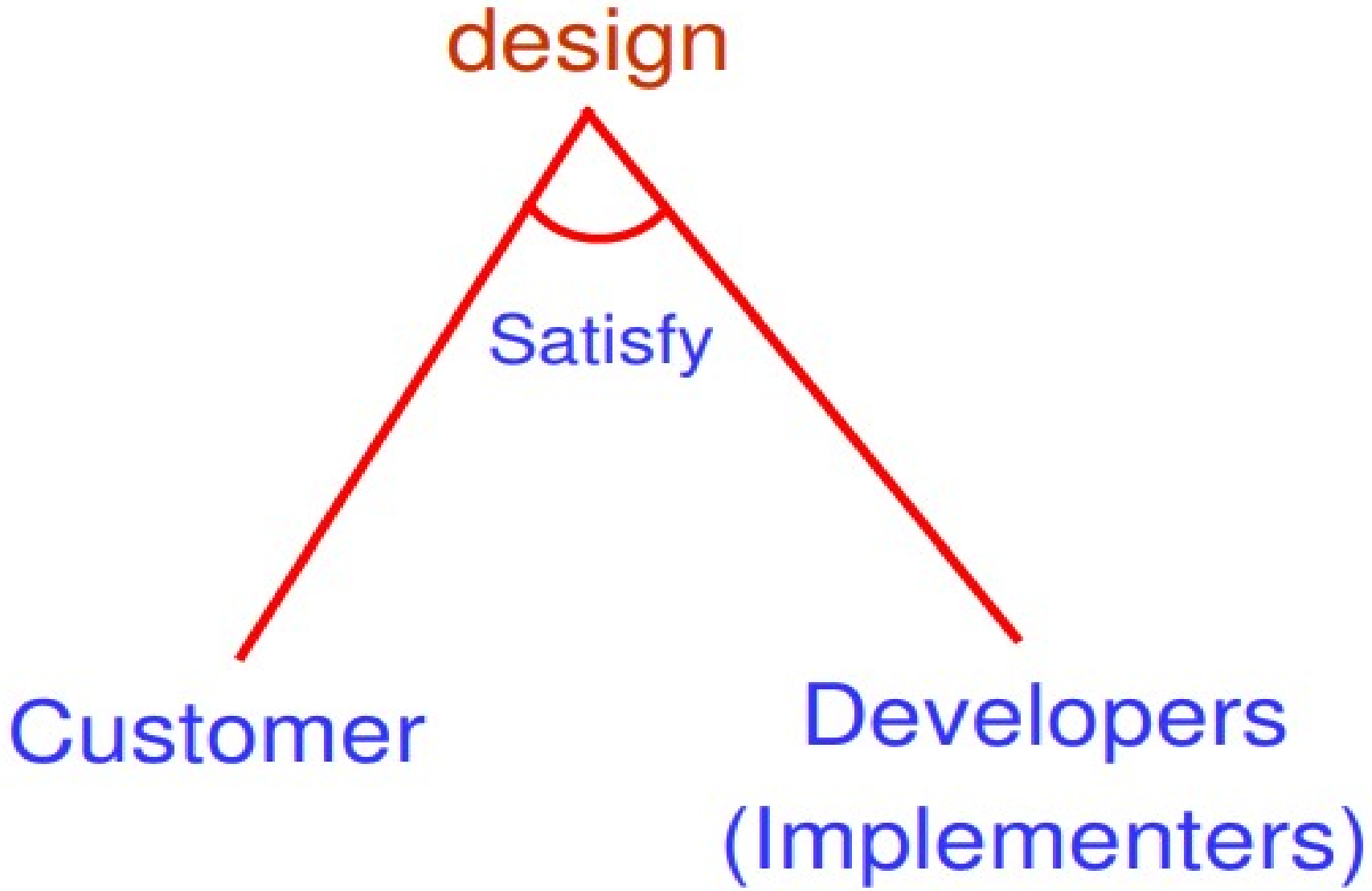
An Overview of System Design

- **System design** is the phase that bridges the **gap between problem domain and the implementation** in a manageable way.
- This phase focuses on the **solution domain**, i.e. *"how to implement?"*
- It is the phase where the **SRS document** is converted into a format that can be implemented and decides **how the system will operate.**
- System design is important for defining the **product and its architecture.**
- It is necessary for the **interfaces, design, data, and modules to satisfy** the system requirements.
- **More creative than analysis**
- **Problem solving activity**

An Overview of System Design

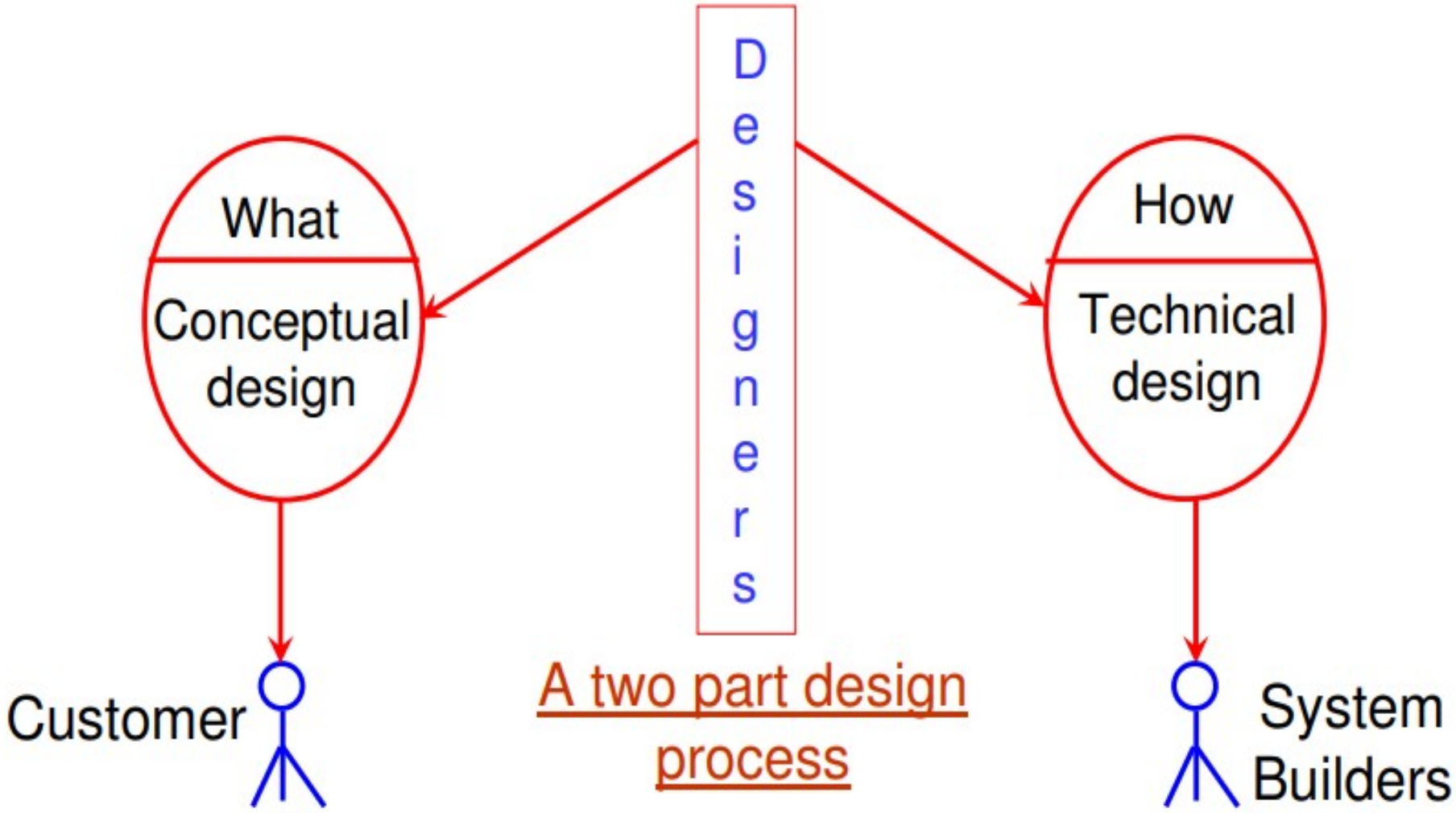


An Overview of System Design



An Overview of System Design

Conceptual Design and Technical Design



An Overview of System Design

- **Conceptual design answers :**
  - Written in simple language i.e. **customer understandable language**.
  - Detailed explanation about **system characteristics.**
  - Describes the **functionality of the system**.
  - It is **independent** of implementation.
  - **Linked** with requirement document

An Overview of System Design

- **Technical design describes :**
  - Hardware configuration
  - Software needs
  - Communication interfaces
  - I/O of the system
  - Software architecture
  - Network architecture
  - Any other thing that translates the requirements in to a Solution to the customer's problem.

An Overview of System Design

- **Elements of a System**
  - **Architecture** - is the conceptual model that defines **the structure, behavior and more views of a system**.
    - We can use flowcharts to represent and illustrate the architecture.
  - **Modules** - This are components that handle one specific tasks in a system.
    - A combination of the modules make up the **system.**
  - **Components** - This provides a particular function or group of related functions.
    - They are made up of **modules**.
  - **Interfaces** - This is the shared boundary across which the components of a the system exchange information and relate.
  - **Data** - This the management of the information and data flow.

System Design Strategies

- In software engineering, a **system design strategy** generally refers to using **system design methodologies** for conceptualizing software requirements.
- A good system design strategy identifies these requirements as challenges and provides the most optimal solutions.
- Then, the strategy incorporates the best approach to implement the solutions.
- The **best system design strategies** always allow **easy manipulation and development of products**.
- A good system design strategy requires foresight and a deeper understanding of **not only the current/active requirements of the software product** but also the **future ones.**

System Design Approach

- **Good system design strategies** take every factor such as **product design, software interface, core architecture, data, and even the modules** into account in order to execute the most effective product development strategy.
- These design modules instruct programmers how to effectively write code for each **subsystem and compile** them together for the **entire system**.
- **There are three approaches to implementing system design. And, they are:**
  - **Structured Design Approach**
  - **Function-oriented Design Approach**
  - **Object-oriented Design Approach**

System Design Approach

Structured Design Approach

- Structured design fundamentally refers to **breaking down problems into multiple well-organized** elements.
- The advantage of using this kind of design strategy is that it allows the **problems to become simpler**.
- This enables problem-solving of the **smaller elements** so that they can fit into the bigger picture.
- The solution modules are structured in a hierarchical order and a manner that there is less **cohesion between** the modules.

System Design Approach

Function-oriented Design Approach

- The function-oriented design strategy is **similar to structured design** but instead **divides the entire system into subsystems** referred to as functions.
- The system is considered to be a map or top-view of all the combined functions.
- However, there is more **information passing between the functions** as compared to structured design while the smaller functions promote abstraction.
- The function-oriented design also allows the program to **operate on input rather than on state**.

System Design Approach

Object-oriented Design Approach

- This design approach is completely different from the other two and focuses **on objects and classes**.
- This kind of strategy revolves around the objects inside the **system and their characteristics**.
- The object-oriented design is based **on identifying objects and grouping them into classes** based on their attributes.

System Design Approach

- **Object-oriented design (OOD)** is the process of using an object-oriented methodology to design a computing system or application.
- This technique enables the implementation of a software solution based on the concepts of objects.
- System design is **decomposed into several activities**, each addressing part of the **overall problem of decomposing** the system:
  - **Identify design goals**
  - **Design the initial subsystem decomposition**
  - **Refine the subsystem decomposition to address the design goals**

Identifying Design Goals

- The goal of **system design** is to allocate the requirements of a large system to hardware and software components.
- In the process of going from the **analysis model to building the system design**, **design goals must be identified.**
  - These will identify **qualities that the system must focus on**,
  - Some design goals will come right from the **non-functional requirements**. Others must be elicited from the **client**
- The overall design goals should be **decided upon *early*** in the system design process, since they will contain **priorities that the rest of the system design must consider**

Identifying Design Goals

Some Criteria for Design Goals

- Performance Criteria
  - **Response time** -- how soon are user requests acknowledged?
  - **Throughput** -- how many tasks can the system accomplish in a certain amount of time?
  - **Memory requirements** -- How much space is required?
- Cost criteria
  - **Development cost** -- intial system development
  - **Deployment cost** -- installation and training
  - **Upgrade cost** -- translating data from previous system, backwards compatibility requirements?
  - **Maintenance cost** -- bug fixes and future enhancements
  - **Administration cost** -- cost of running the system

Identifying Design Goals

Some Criteria for Design Goals

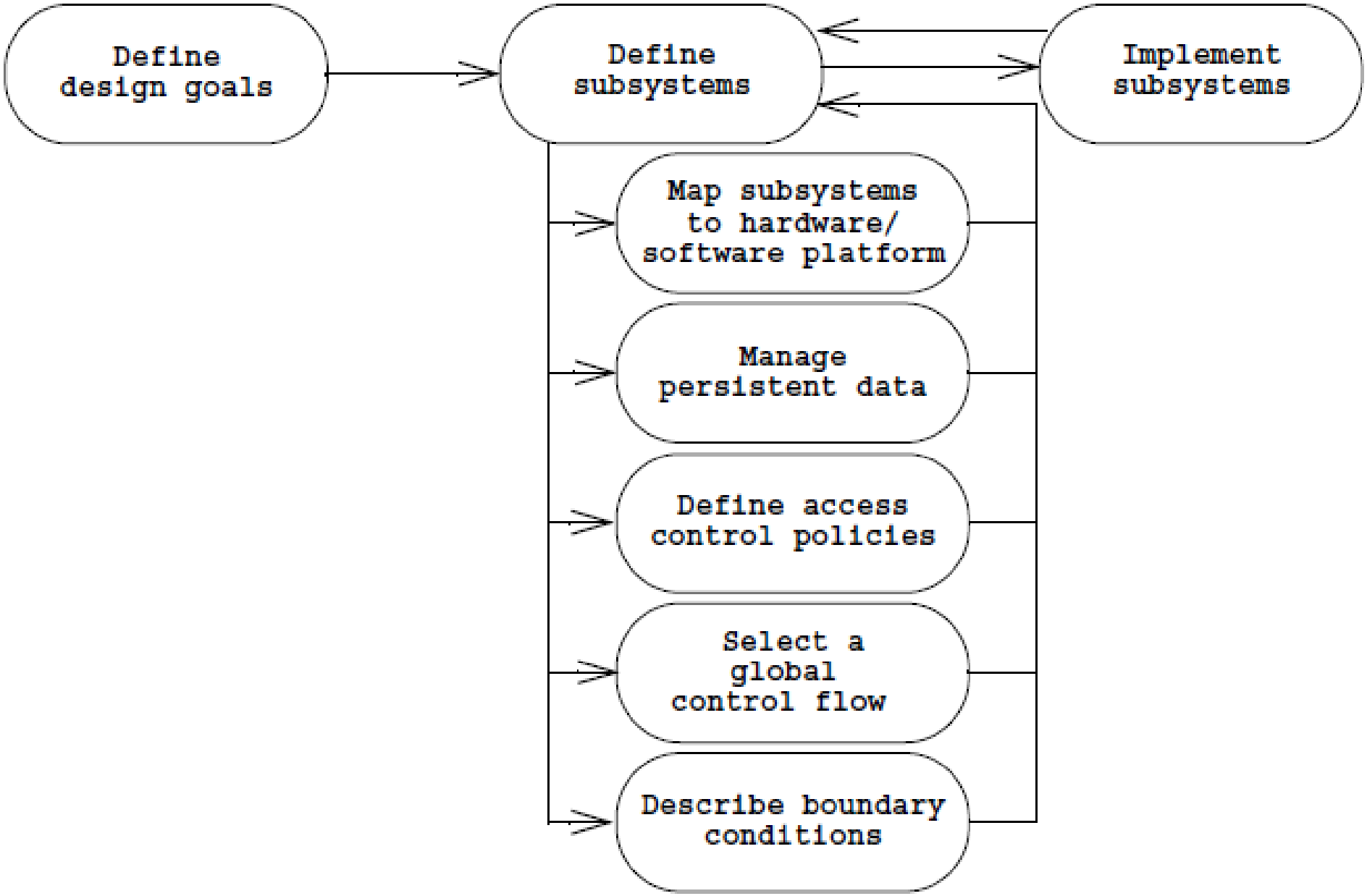
- Dependability Criteria
  - Robustness -- how does the system cope with invalid user input? **how much software is tolerant to faults**
  - **Reliability** -- difference between specified and observed behavior
  - **Availability** -- percentage of time the system can be used to accomplish normal user tasks
  - **Fault tolerance** -- how does it handle erroneous conditions?
  - **Security** -- how does it withstand malicious attacks?
  - **Safety** -- ability to avoid endangering human lives, even through errors and failures
    - This one is *very* important!
    - Ex: onboard system of an airplane, software running an x-ray machine, etc
- Cost criteria
  - Development cost -- intial system development
  - Deployment cost -- installation and training
  - Upgrade cost -- translating data from previous system, backwards compatibility requirements?
  - Maintenance cost -- bug fixes and future enhancements
  - Administration cost -- cost of running the system
- Maintenance criteria
  - Extensibility -- how easy to add functionlity
  - Modifiability -- how easy to change functinality
  - Adaptability -- how easy to adapt to different applcation domain
  - Portability -- how easy to port to a differentcomputer platform
  - Readability -- how easy to understand the coed
  - Tracability of requirements -- mapping of code to requirements
- End-user criteria
  - Utility -- how well does it support the end user's work?
  - Usability -- how easy is it to use?

Identifying Design Goals

Some Criteria for Design Goals

- Maintenance criteria
  - **Extensibility** -- how easy to add functionality
  - **Modifiability** -- how easy to change functionality
  - **Adaptability** -- how easy to adapt to different application domain
  - **Portability** -- how easy to port to a different computer platform
  - **Readability** -- how easy to understand the coed
  - **Tracability of requirements** -- mapping of code to requirements
- End-user criteria
  - **Utility** -- how well does it support the end user's work?
  - Usability -- how easy is it to use?

Identifying Design Goals



Identifying Design Goals

- *In particular, they need to address the following issues:*

- ***Hardware/software mapping**:*
  - *What is the hardware configuration of the system?*
  - *Which node is responsible, for which functionality?*
  - *How is communication between nodes realized?*
  - *Which services are realized using existing software components?*
  - *How are these components encapsulated?*

- *Addressing hardware/software mapping issues often leads to*
  - *definition of additional subsystems dedicated to moving data from one node to another,*
  - *dealing with concurrency, and reliability issues.*

Identifying Design Goals

- ***Off-the-shelf components*** *enable developers to realize complex services more economically.*
  - *User interface packages*
  - *DBMS are prime examples of off-the-shelf components.*

- *Components, however, should be encapsulated to minimize dependencies on a particular component:*
  - *A competing vendor may come up with a better component.*

Identifying Design Goals

- ***Data management:***
  - *Which data need to be persistent?*
  - *Where should persistent data be stored?*
  - *How are they accessed?*
- ***Persistent data represents a bottleneck in the system*** *on many different fronts: Most functionality in system is concerned with creating or manipulating persistent data.*
  - *For this reason, access to the data should be* ***fast and reliable.***
  - *If retrieving data is slow, the whole* ***system will be slow.***
  - *If data corruption is likely, complete system* ***failure is likely.***
- *These issues need to be addressed consistently at the system level. Often, this leads to selection of* ***DBMS and of an additional subsystem dedicated to management of persistent data****.*

Identifying Design Goals

- ***Access control:***

Access control and security are system-wide issues. The **access control policy used to specify who can and cannot access certain data should be the same *across all subsystems*.**

  - *Who can access, which data?*
  - *Can access control change dynamically?*
  - *How is access control specified and realized?*

- ***Control flow:***

  - *How does the system sequence operations?*
  - *Is the system event driven?*
  - *Can it handle more than one user interaction at a time?*

- *The choice of control flow has an impact on the interfaces of subsystems.*

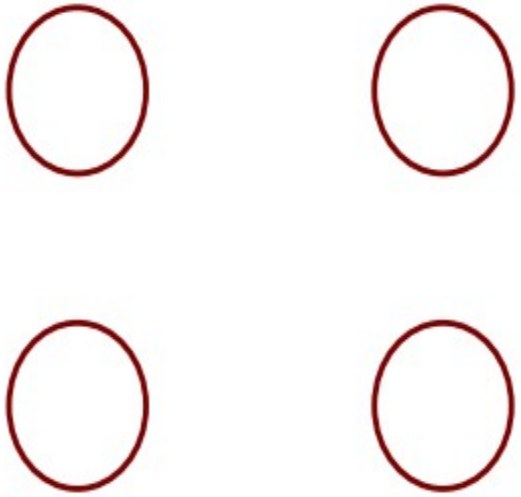
Identifying Design Goals

- ***Boundary conditions:***
  - *How is the system initialized?*
  - *How is it shut down?*
  - *How are exceptional cases detected and handled?*
- *System initialization and shutdown often represent the larger part of the **complexity of a system**, especially in a distributed environment.*
- *Initialization, shutdown, and exception handling have an impact on the interface of all subsystems.*

modularization

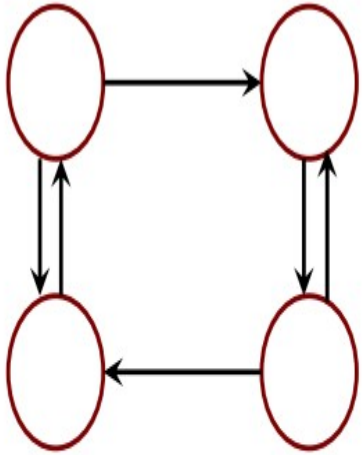
- **Modularization:** is the process of **dividing a software system into multiple independent modules** where each module works independently.
- There are many advantages of Modularization in software engineering. Some of these are given below:
  - **Easy to understand the system.**
  - **System maintenance is easy.**
  - **A module can be used many times as their requirements. No need to write it again and again.**
- Both coupling and cohesion are important factors in determining the **maintainability, scalability, and reliability of a software system**.
- **High coupling and low cohesion** can make a system difficult to change and test, while low coupling and high cohesion make a system easier **to maintain and improve.**

Coupling



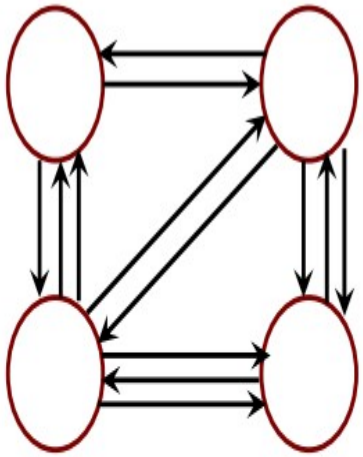
(Uncoupled : no dependencies)

(a)



Loosely coupled:
some dependencies

(B)

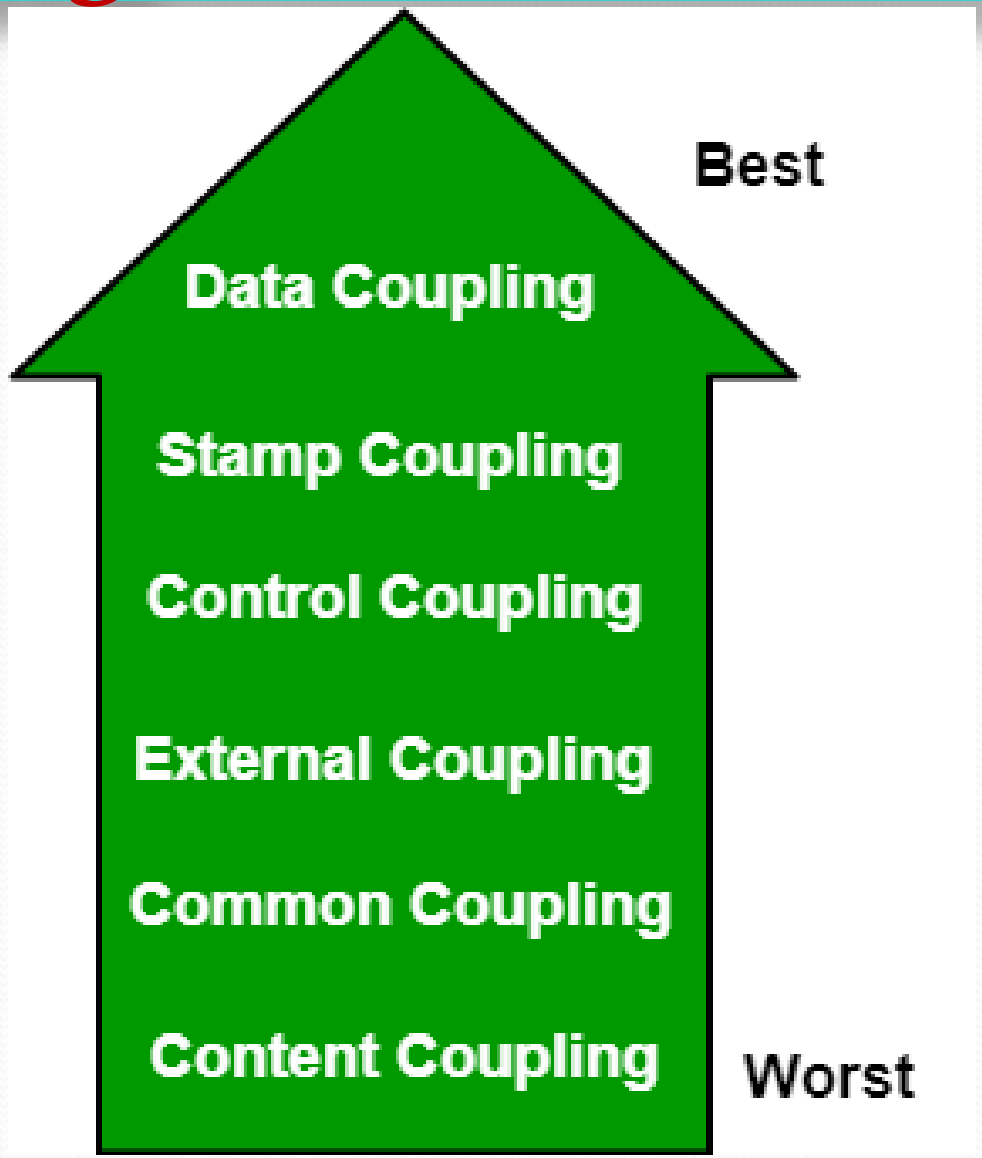


Highly coupled:
many dependencies

(C)

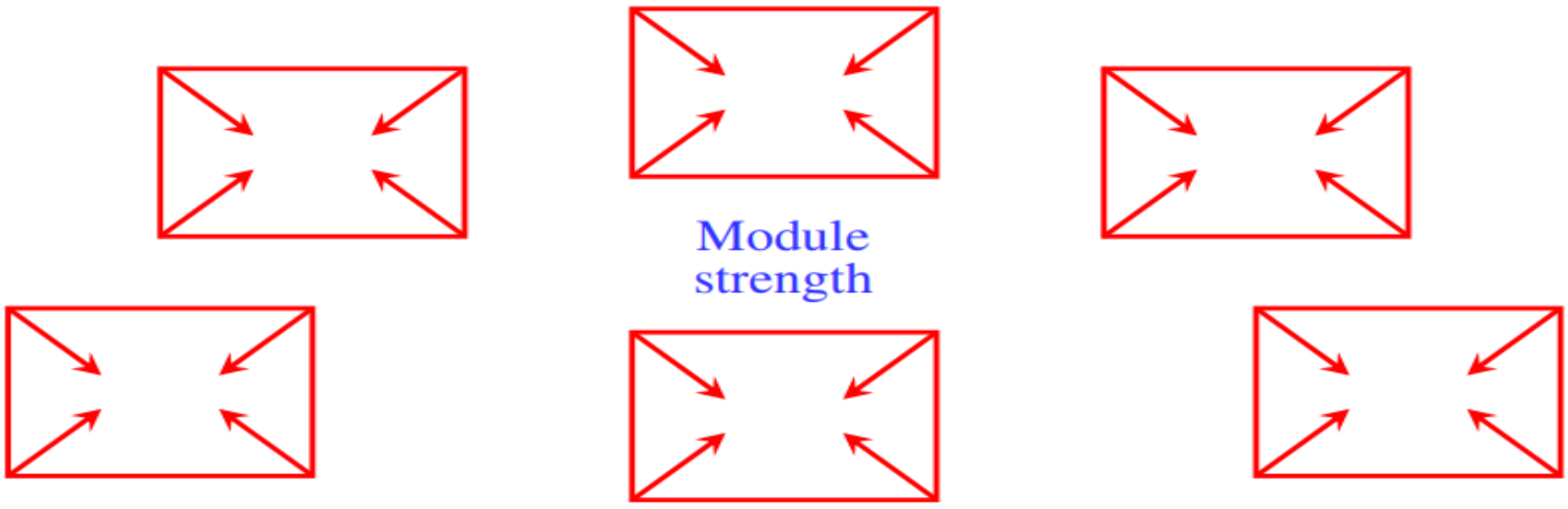
Coupling: Coupling is the measure of the degree of interdependence between the modules. A good software will have low coupling.

Coupling


Best
Data Coupling
Stamp Coupling
Control Coupling
External Coupling
Common Coupling
Content Coupling
Worst

Cohesion

- **Cohesion:** Cohesion is a measure of the degree to which the elements of the module are functionally related.
- It is the degree to which all elements directed towards performing a single task are contained in the component.
- Basically, cohesion is the internal glue that keeps the module together. A good software design will have high cohesion.

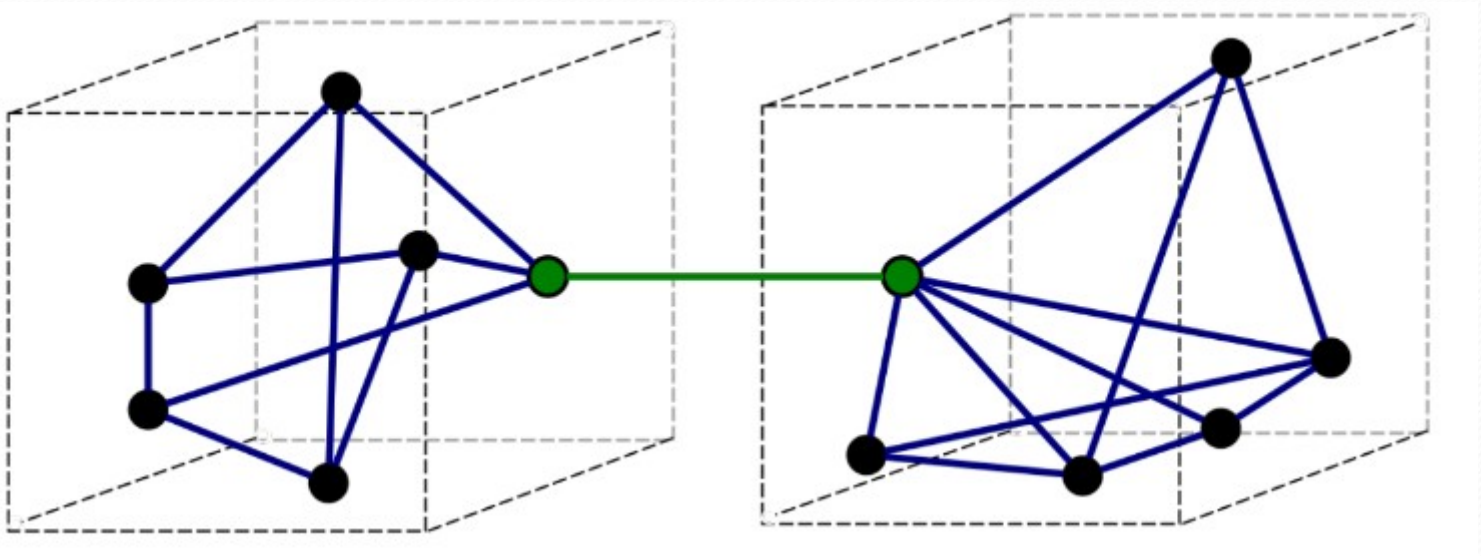


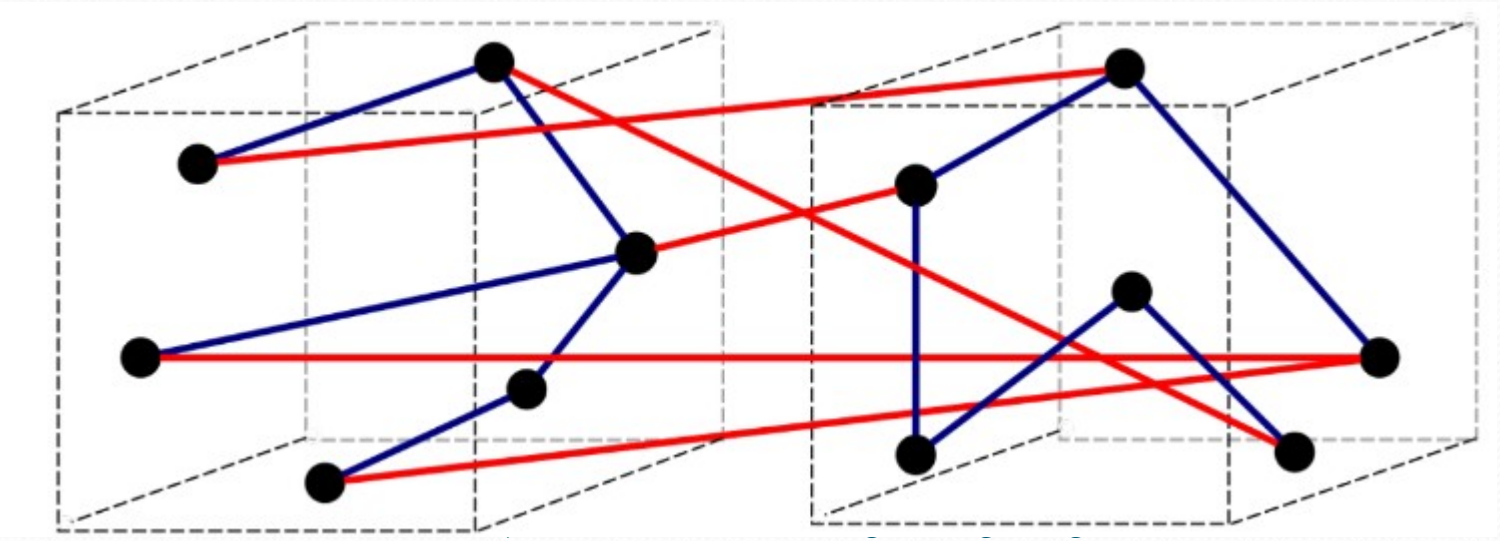
Cohesion

Functional Cohesion	Best (high)
Sequential Cohesion	
Communicational Cohesion	
Procedural Cohesion	
Temporal Cohesion	
Logical Cohesion	
Coincidental Cohesion	Worst (low)

differentiate high coupling and low cohesion

Exercise





Exercise

- *Explain in detail the similarity and difference between the **Six coupling** types.*
- *Explain in detail the similarity and difference between the **Seven Cohesion types**.*

Reviewing System Design

- *Ensure that the system design model is* ***correct, complete, consistent, realistic, and readable.***
- *The system design model is* ***correct*** *if the* ***analysis model can be mapped to the system design model****. You should ask the following questions to determine if the system design is correct:*
  - *Can every subsystem be traced back to a* ***use case or a nonfunctional requirement****?*
  - *Can every use case be mapped to a* ***set of subsystems****?*
  - *Can every design* ***goal be traced back to a nonfunctional requirement?***
  - *Is every nonfunctional requirement addressed in the* ***system design model?***
  - *Does each actor have an* ***access policy****?*
  - *Is it* ***consistent with the nonfunctional security*** *requirement?*

Reviewing System Design

- *The model is **complete** if **every requirement and every system design issue** has been addressed.*
- *You should ask the following questions to determine if the system design is complete:*
  - *Have the **boundary conditions been handled**?*
  - *Was there a walkthrough of the use cases to identify **missing functionality in the system design**?*
  - *Have all use cases been **examined and assigned a control object?***
  - *Have all aspects of system design (i.e., hardware allocation, persistent storage, access control, legacy code, boundary conditions) been addressed?*
  - *Do all subsystems have definitions?*

Reviewing System Design

- *The model is **consistent** if it does not contain any contradictions. You should ask the following questions to determine if a system design is consistent:*
  - *Are conflicting design goals prioritized?*
  - *Are there design goals that violate a nonfunctional requirement?*
  - *Are there multiple subsystems or classes with the same name?*
  - *Are collections of objects exchanged among subsystems in a consistent manner?*

Reviewing System Design

- *The model is **realistic** if the **corresponding system can be implemented.** You ask the following questions to determine if a system design is realistic:*
  - *Are there any **new technologies or components** in the system?*
  - *Were there any studies to evaluate the **appropriateness or robustness of these technologies** or components?*
  - *Have performance and reliability requirements been reviewed in the context of the subsystem decomposition? For example, is there a **network connection on the critical path of the system**?*
  - *Have concurrency issues **(e.g., contention, deadlocks, mutual exclusion) been addressed**?*

Reviewing System Design

- *The model is **readable** if **developers not involved** in the system design can **understand the model**.*
- *You should ask the following questions to ensure that the system design is readable:*
  - *Are **subsystem names understandable**?*
  - *Do entities (e.g., **subsystems, classes, operations**) with similar names denote similar phenomena?*
  - *Are all entities described at the **same level of detail**?*

Managing System Design

- ***Documenting system design***
- *System design is documented in the System Design Document (SDD).*
- *It describes design goals set by the project, the subsystem decomposition* ***(with UML class diagrams), the hardware/software mapping (with UML deployment diagrams), the data management, the access control, control flow mechanisms, and boundary conditions****.*
- *The SDD is used to define interfaces between teams of developers and as reference when architecture-level decisions need to be revisited.*
- *The audience for the SDD includes the* ***project management, the system architects (i.e., the developers who participate in the system design), and the developers who design and implement each subsystem.*** *The following is an example template for a SDD:*

System Design Document (SDD) Structure

1. Introduction
   - 1.1 Purpose of the system
   - 1.2 Design goals
   - 1.3 Definitions, acronyms, and abbreviations
   - 1.4 References
   - 1.5 Overview
2. Current software architecture
3. Proposed software architecture
   - 3.1 Overview
   - 3.2 Subsystem decomposition
   - 3.3 Hardware/software mapping
   - 3.4 Persistent data management
   - 3.5 Access control and security
   - 3.6 Global software control
   - 3.7 Boundary conditions
4. Subsystem services

Glossary

Managing System Design

1. Introduction → *brief overview of the software architecture and the design goals*
   - 1.1 Purpose of the system
   - 1.2 Design goa1s
   - 1.3 Definitions, acronyms, and abbreviations
   - 1.4 References → *References to other documents and traceability information (e.g., related requirements analysis document, references to existing systems, constraints impacting the software architecture).*
   - 1.5 Overview
2. Current software architecture
3. Proposed software architecture
   - 3.1 Overview
   - 3.2 Subsystem decomposition
   - 3.3 Hardware/software mapping
   - 3.4 Persistent data management
   - 3.5 Access control and security
   - 3.6 Global software control
   - 3.7 Boundary conditions
4. Subsystem services

Glossary

Managing System Design

1. Introduction
   - 1.1 Purpose of the system
   - 1.2 Design goa1s
   - 1.3 Definitions, acronyms, and abbreviations
   - 1.4 References
   - 1.5 Overview
2. Current software architecture
3. Proposed software architecture
   - 3.1 Overview
   - 3.2 Subsystem decomposition
   - 3.3 Hardware/software mapping
   - 3.4 Persistent data management
   - 3.5 Access control and security
   - 3.6 Global software control
   - 3.7 Boundary conditions
4. Subsystem services

Glossary

the architecture of the system being replaced. If there is no previous system, this section can be replaced by a survey of current architectures for similar systems

The purpose of this section is to make explicit the background information that system architects used, their assumptions, and common issues the new system will address.

Managing System Design

1. Introduction
    1.1 Purpose of the system
    1.2 Design goaıs
    1.3 Definitions, acronyms, and abbreviations
    1.4 References
    1.5 Overview
2. Current software architecture
3. Proposed software architecture
    3.1 Overview
    3.2 Subsystem decomposition
    3.3 Hardware/software mapping
    3.4 Persistent data management
    3.5 Access control and security
    3.6 Global software control
    3.7 Boundary conditions
4. Subsystem services

Glossary

documents the system design model of the new system

presents a bird's-eye view of the software architecture and briefly describes the assignment of functionality to each subsystem

describes the decomposition into subsystems and the responsibilities of each.
This is the main product of system design

Managing System Design

1. Introduction
   - 1.1 Purpose of the system
   - 1.2 Design goa1s
   - 1.3 Definitions, acronyms, and abbreviations
   - 1.4 References
   - 1.5 Overview
2. Current software architecture
3. Proposed software architecture
   - 3.1 Overview
   - 3.2 Subsystem decomposition
   - 3.3 Hardware/software mapping
   - 3.4 Persistent data management
   - 3.5 Access control and security
   - 3.6 Global software control
   - 3.7 Boundary conditions
4. Subsystem services

Glossary

describes how subsystems are assigned to hardware and off-the-shelf components. It also lists the issues introduced by multiple nodes and software reuse

describes the persistent data stored by the system and the data management infrastructure required for it. This section typically includes the description of data schemes, the selection of a database, and the description of the encapsulation of the database

Managing System Design

1. Introduction
   - 1.1 Purpose of the system
   - 1.2 Design goa1s
   - 1.3 Definitions, acronyms, and abbreviations
   - 1.4 References
   - 1.5 Overview
2. Current software architecture
3. Proposed software architecture
   - 3.1 Overview
   - 3.2 Subsystem decomposition
   - 3.3 Hardware/software mapping
   - 3.4 Persistent data management
   - 3.5 Access control and security
   - 3.6 Global software control
   - 3.7 Boundary conditions
4. Subsystem services

Glossary

describes the user model of the system in terms of an access matrix.
This section also describes security issues, such as the selection of an authentication mechanism, the use of encryption, and the management of keys

Managing System Design

1. Introduction
   - 1.1 Purpose of the system
   - 1.2 Design goa1s
   - 1.3 Definitions, acronyms, and abbreviations
   - 1.4 References
   - 1.5 Overview
2. Current software architecture
3. Proposed software architecture
   - 3.1 Overview
   - 3.2 Subsystem decomposition
   - 3.3 Hardware/software mapping
   - 3.4 Persistent data management
   - 3.5 Access control and security
   - 3.6 Global software control
   - 3.7 Boundary conditions
4. Subsystem services

Glossary

describes how the global software control is implemented.
In particular, this section should describe how requests are initiated and how subsystems synchronize.
This section should list and address synchronization and concurrency issues

Managing System Design

1. Introduction
    1.1 Purpose of the system
    1.2 Design goa1s
    1.3 Definitions, acronyms, and abbreviations
    1.4 References
    1.5 Overview
2. Current software architecture
3. Proposed software architecture
    3.1 Overview
    3.2 Subsystem decomposition
    3.3 Hardware/software mapping
    3.4 Persistent data management
    3.5 Access control and security
    3.6 Global software control
    3.7 Boundary conditions
4. Subsystem services

Glossary

describes the start-up, shutdown, and error behaviour of the system. (If new use cases are discovered for system administration, these should be included in the requirements analysis document, not in this section.)

Managing System Design

1. Introduction
   - 1.1 Purpose of the system
   - 1.2 Design goa1s
   - 1.3 Definitions, acronyms, and abbreviations
   - 1.4 References
   - 1.5 Overview
2. Current software architecture
3. Proposed software architecture
   - 3.1 Overview
   - 3.2 Subsystem decomposition
   - 3.3 Hardware/software mapping
   - 3.4 Persistent data management
   - 3.5 Access control and security
   - 3.6 Global software control
   - 3.7 Boundary conditions
4. Subsystem services

Glossary

- *describes the services provided by each subsystem in terms of operations.*
- *Although this section is usually empty or incomplete in the first versions of the SDD, this section serves as a reference for teams for the boundaries between their subsystems.*
- *The interface of each subsystem is derived from this section and detailed in the Object Design Document*

Thank you for ur attention

Questions?